

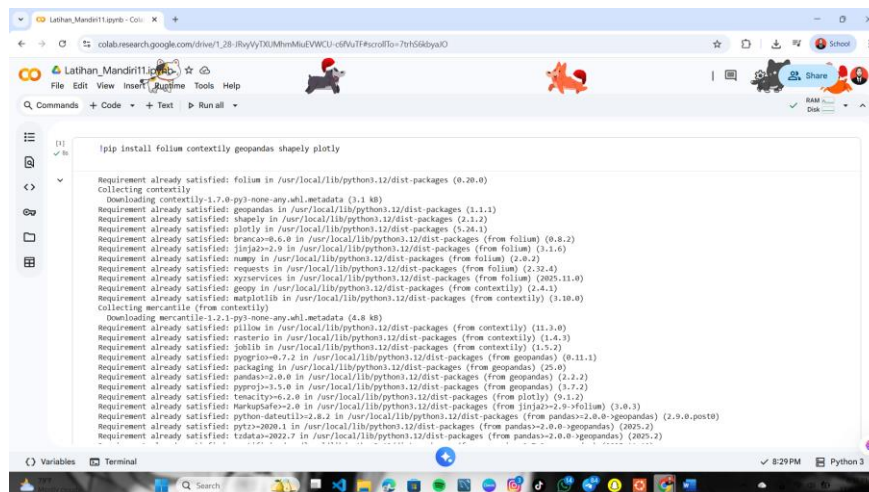
Tugas 11: Tugas Praktikum Mandiri 11 – Machine Learning

Yurida Yahsyia 1 - 0110224100 1*

¹ Teknik Informatika, STT Terpadu Nurul Fikri, Depok

*E-mail: 0110224100@student.nurulfikri.ac.id – email mahasiswa 1

1. Tugas Mandiri 11

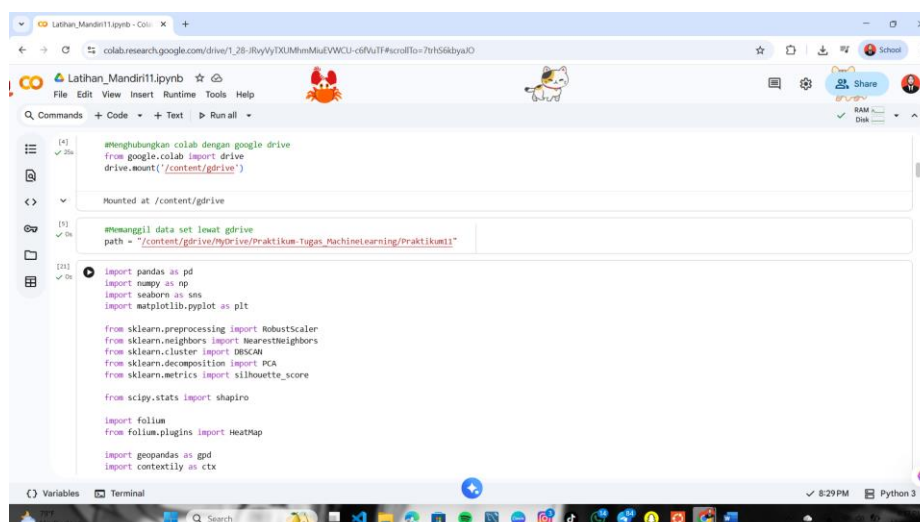


```
!pip install folium contextily geopandas shapely plotly
```

Requirement already satisfied: folium in /usr/local/lib/python3.12/dist-packages (0.20.0)
Collecting contextily
 Downloading contextily-1.7.0-py3-none-any.whl.metadata (3.1 kB)
Requirement already satisfied: geopandas in /usr/local/lib/python3.12/dist-packages (0.11.1)
Requirement already satisfied: shapely in /usr/local/lib/python3.12/dist-packages (1.8.2)
Requirement already satisfied: plotly in /usr/local/lib/python3.12/dist-packages (5.10.0)
Requirement already satisfied: branca<0.6.0 in /usr/local/lib/python3.12/dist-packages (from folium) (0.5.2)
Requirement already satisfied: Jinja2<2.9 in /usr/local/lib/python3.12/dist-packages (from folium) (3.1.6)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from folium) (2.32.4)
Requirement already satisfied: xyzservices in /usr/local/lib/python3.12/dist-packages (from folium) (2025.11.0)
Requirement already satisfied: GeoPy in /usr/local/lib/python3.12/dist-packages (from contextily) (2.4.1)
Requirement already satisfied: mercantile in /usr/local/lib/python3.12/dist-packages (from contextily) (3.10.0)
Collecting mercantile
 Downloading mercantile-1.2.1-py3-none-any.whl.metadata (4.8 kB)
Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages (from contextily) (11.3.0)
Requirement already satisfied: rasterio in /usr/local/lib/python3.12/dist-packages (from contextily) (1.4.5)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from contextily) (1.5.2)
Requirement already satisfied: pyproj<0.7.2 in /usr/local/lib/python3.12/dist-packages (from geopandas) (0.11.1)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from geopandas) (25.0)
Requirement already satisfied: pandas<2.0.0 in /usr/local/lib/python3.12/dist-packages (from geopandas) (2.2.2)
Requirement already satisfied: pyproj<3.5.0 in /usr/local/lib/python3.12/dist-packages (from geopandas) (3.7.2)
Requirement already satisfied: tenacity<6.2.0 in /usr/local/lib/python3.12/dist-packages (from plotly) (9.1.2)
Requirement already satisfied: MarkupSafe<2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2<2.9>from folium) (3.0.3)
Requirement already satisfied: python-dateutil<2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas<2.0.0>from geopandas) (2.9.0.post0)
Requirement already satisfied: pytz<2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas<2.0.0>from geopandas) (2020.2)
Requirement already satisfied: tzdata<2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas<2.0.0>from geopandas) (2022.2)

1. !pip install folium contextily geopandas shapely plotly

Langkah ini memastikan bahwa lingkungan kerja siap dengan semua alat yang diperlukan untuk memuat, menganalisis, dan memvisualisasikan data geografis.



```
from google.colab import drive
drive.mount('/content/gdrive')

#memanggil data set lewat gdrive
path = "/content/gdrive/MyDrive/Praktikum-Tugas-MachineLearning/Praktikum11"

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.preprocessing import RobustScaler
from sklearn.neighbors import NearestNeighbors
from sklearn.cluster import DBSCAN
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score

from scipy.stats import shapiro

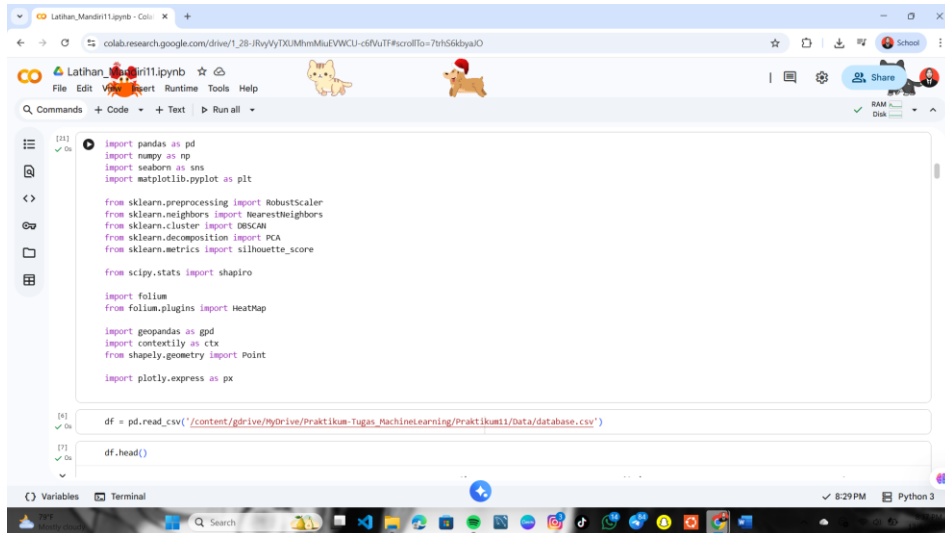
import folium
from folium.plugins import HeatMap

import geopandas as gpd
import contextily as ctx
```

1. from google.colab import drive drive.mount('/content/gdrive')

Menghubungkan collab dengan google drive

2. **path = "/content/gdrive/MyDrive/Praktikum-Tugas Machinelearning/Praktikum11"**
Memanggil dataset lewat gdrive



```
[1] ✓ On
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.preprocessing import RobustScaler
from sklearn.neighbors import NearestNeighbors
from sklearn.cluster import DBSCAN
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score

from scipy.stats import shapiro

import folium
from folium.plugins import HeatMap

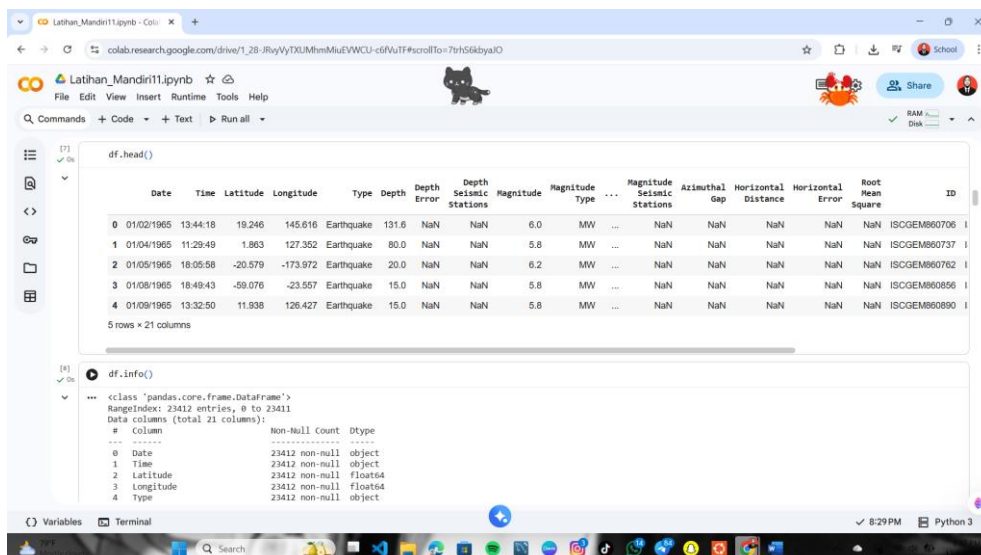
import geopandas as gpd
import contextily as ctx
from shapely.geometry import Point

import plotly.express as px

[4] ✓ On
df = pd.read_csv('/content/gdrive/MyDrive/Praktikum-Tugas Machinelearning/Praktikum11/Data/database.csv')

[7] ✓ On
df.head()
```

1. Seluruh kode import diatas untuk mengimpor Pustaka
2. **df = pd.read_csv('/content/gdrive/MyDrive/Praktikum-Tugas Machinelearning/Praktikum11/Data/database.csv')**
menyimpan seluruh isi file CSV ke dalam memori komputer, di dalam objek df



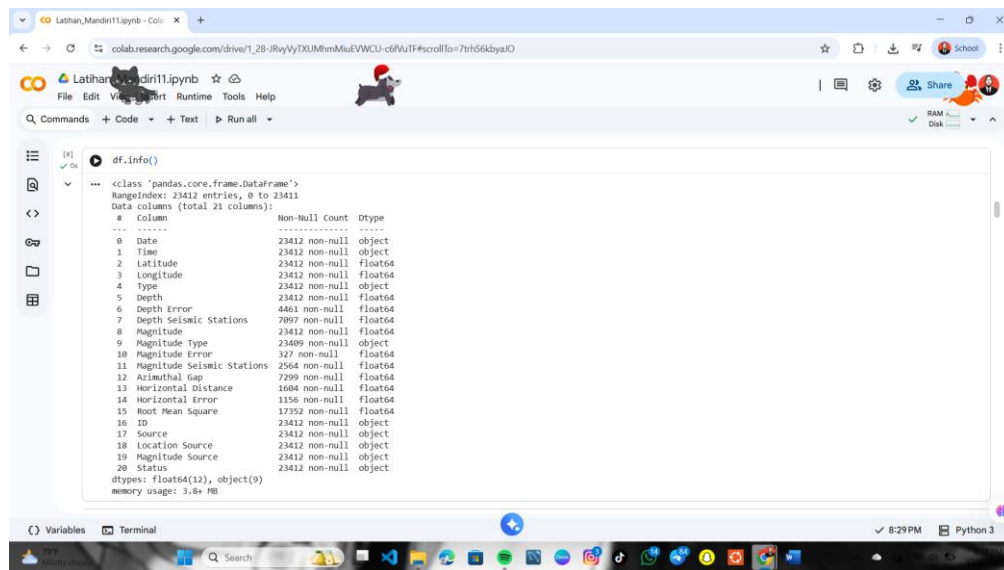
```
[7] ✓ On
df.head()

   Date      Time  Latitude  Longitude  Type  Depth  Depth Error  Depth Seismic Stations  Magnitude  Magnitude Type  ...  Magnitude Seismic Stations  Azimuthal Gap  Horizontal Distance  Horizontal Error  Root Mean Square  ID
0  01/02/1965  13:44:18   19.246   145.616  Earthquake  131.6    NaN      NaN      6.0    MW  ...    NaN      NaN      NaN      NaN      NaN  ISCGEM860706  I
1  01/04/1965  11:29:49    1.863   127.352  Earthquake   80.0    NaN      NaN      5.8    MW  ...    NaN      NaN      NaN      NaN      NaN  ISCGEM860737  I
2  01/05/1965  18:05:58  -20.579   -173.972  Earthquake   20.0    NaN      NaN      6.2    MW  ...    NaN      NaN      NaN      NaN      NaN  ISCGEM860762  I
3  01/08/1965  18:49:43  -59.076   -23.557  Earthquake   15.0    NaN      NaN      5.8    MW  ...    NaN      NaN      NaN      NaN      NaN  ISCGEM860856  I
4  01/09/1965  13:32:50   11.938   126.427  Earthquake   15.0    NaN      NaN      5.8    MW  ...    NaN      NaN      NaN      NaN      NaN  ISCGEM860890  I
5 rows x 21 columns

[8] ✓ On
df.info()

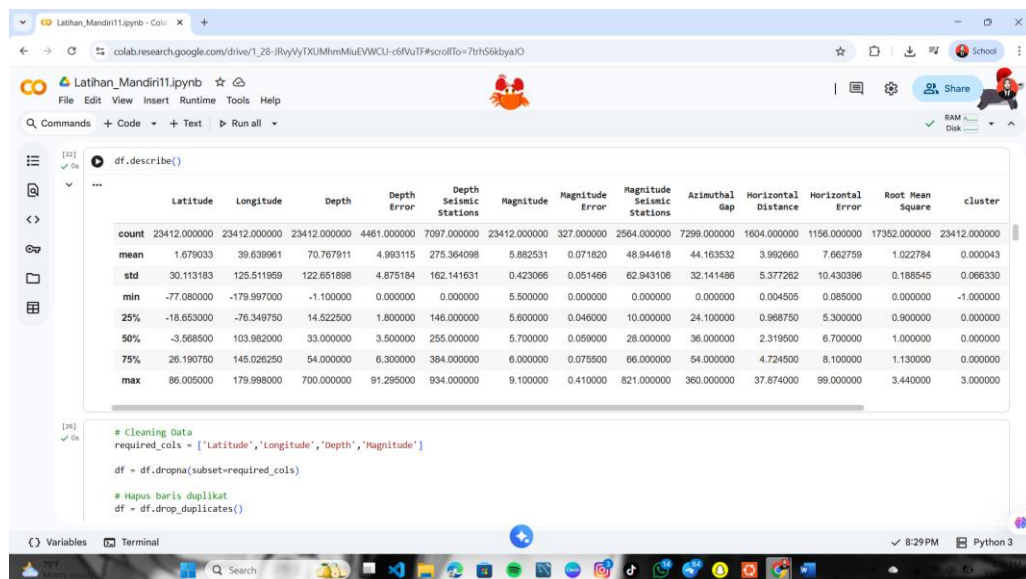
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 23412 entries, 0 to 23411
Data columns (total 21 columns):
 #   Column              Non-Null Count  Dtype  
---  --
 0   Date                23412 non-null  object  
 1   Time                23412 non-null  object  
 2   Latitude            23412 non-null  float64  
 3   Longitude           23412 non-null  float64  
 4   Type                23412 non-null  object
```

1. **df.head()**
Menampilkan 5 baris pertama dari DataFrame df



```
df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 23412 entries, 0 to 23411
Data columns (total 21 columns):
 #   Column              Non-Null count  Dtype
---  --
 0   Date                23412 non-null object
 1   Time                23412 non-null object
 2   Latitude            23412 non-null float64
 3   Longitude           23412 non-null float64
 4   Type                23412 non-null object
 5   Depth              23412 non-null float64
 6   Depth Error         4461 non-null  float64
 7   Depth Seismic Stations 7897 non-null  float64
 8   Magnitude           23412 non-null float64
 9   Magnitude Type      23409 non-null object
10   Magnitude Error     327 non-null   float64
11   Magnitude Seismic Stations 2564 non-null float64
12   Azimuthal Gap       7299 non-null  float64
13   Horizontal Distance 1684 non-null  float64
14   Horizontal Error    1156 non-null  float64
15   Root Mean Square    17352 non-null float64
16   ID                  23412 non-null object
17   Source              23412 non-null object
18   Location Source     23412 non-null object
19   Magnitude Source    23412 non-null object
20   Status              23412 non-null object
dtypes: float64(12), object(9)
memory usage: 3.8+ MB
```

1. **df.info()**
Memberikan ringkasan komprehensif dari DataFrame df.



```
df.describe()

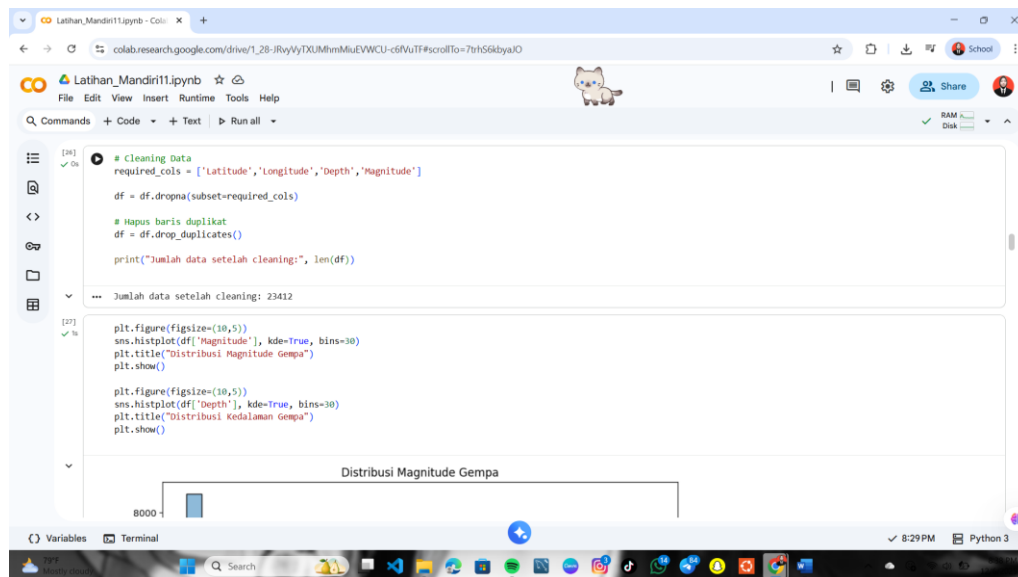
```

	Latitude	Longitude	Depth	Depth Error	Depth Seismic Stations	Magnitude	Magnitude Error	Magnitude Seismic Stations	Azimuthal Gap	Horizontal Distance	Horizontal Error	Root Mean Square	cluster
count	23412.000000	23412.000000	23412.000000	4461.000000	7897.000000	23412.000000	327.000000	2564.000000	7299.000000	1684.000000	1156.000000	17352.000000	23412.000000
mean	1.679033	39.639961	70.767911	4.993115	275.364098	5.882531	0.071820	48.944618	44.163532	3.992660	7.662759	1.022784	0.000043
std	30.113183	125.511959	122.651898	4.875184	162.141631	0.423066	0.051466	62.943106	32.141486	5.377262	10.430396	0.188545	0.066330
min	-77.080000	-179.997000	-1.100000	0.000000	0.000000	5.500000	0.000000	0.000000	0.000000	0.004505	0.085000	0.000000	-1.000000
25%	-18.653000	-76.349750	14.522500	1.800000	146.000000	5.800000	0.046000	10.000000	24.100000	0.968750	5.300000	0.900000	0.000000
50%	-3.568500	103.982000	33.000000	3.500000	255.000000	5.700000	0.059000	28.000000	36.000000	2.319500	6.700000	1.000000	0.000000
75%	26.190750	145.026250	64.000000	6.300000	384.000000	6.000000	0.075500	66.000000	54.000000	4.724500	8.100000	1.130000	0.000000
max	86.005000	179.998000	700.000000	91.295000	934.000000	9.100000	0.410000	821.000000	360.000000	37.874000	99.000000	3.440000	3.000000

```
# Cleaning Data
required_cols = ['Latitude', 'Longitude', 'Depth', 'Magnitude']
df = df.dropna(subset=required_cols)

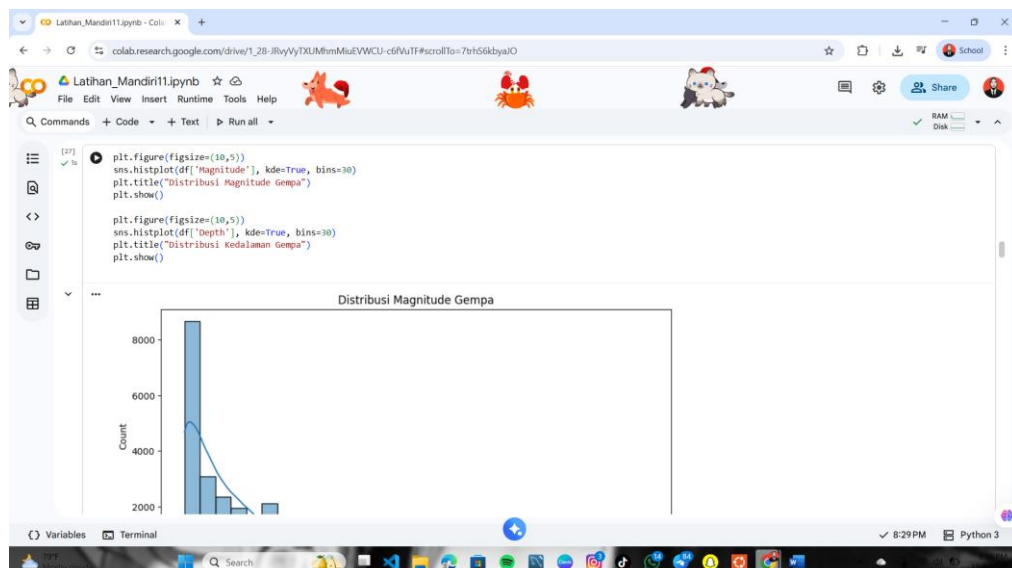
# Hapus baris duplikat
df = df.drop_duplicates()
```

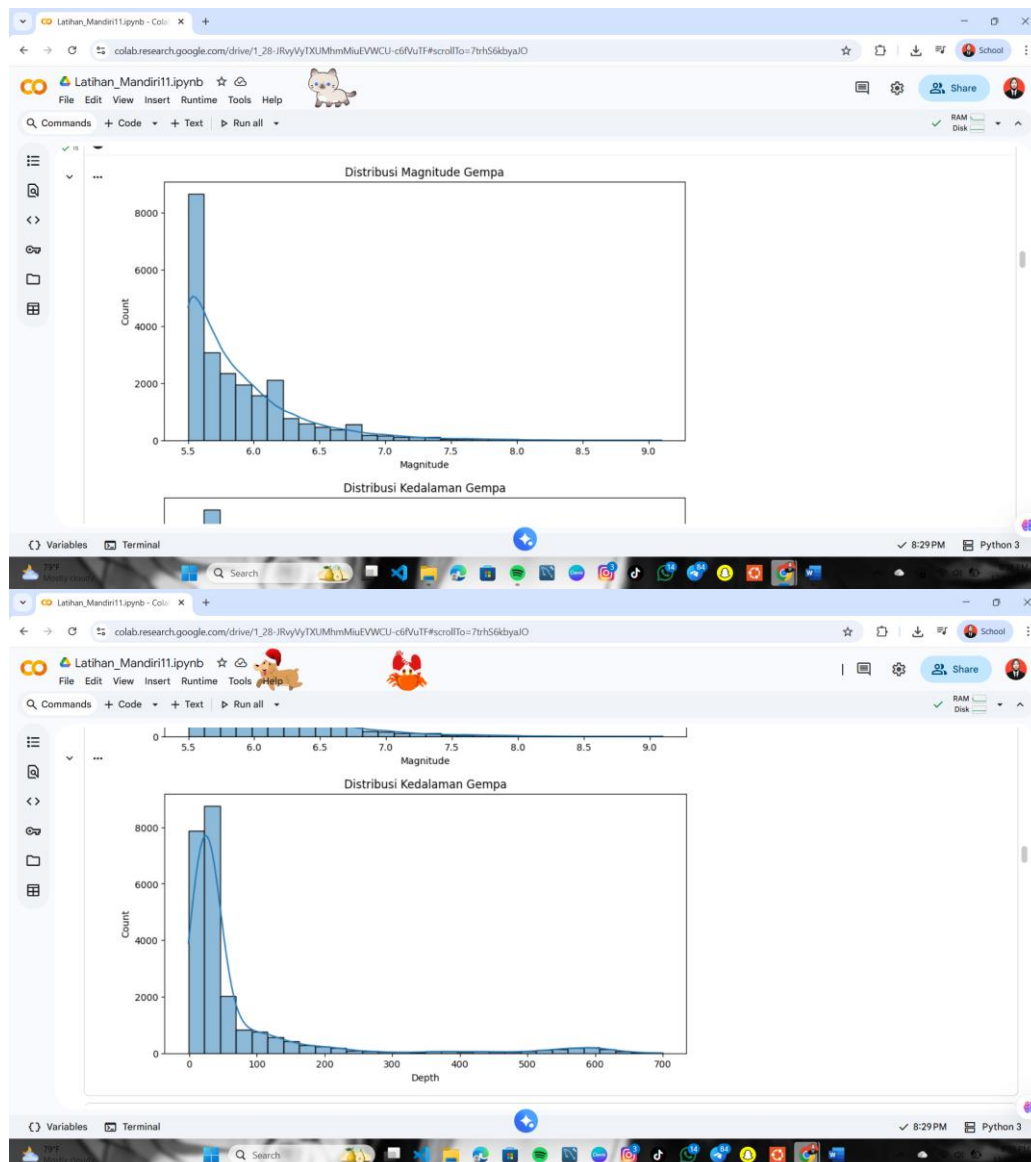
1. **df.describe()**
menghasilkan statistik deskriptif dari DataFrame.



1. **print("Jumlah data setelah cleaning:", len(df))**
untuk mencetak dan menampilkan jumlah total baris yang tersisa dalam DataFrame df setelah proses pembersihan (menghapus baris dengan nilai kosong pada kolom kunci dan menghapus duplikat).

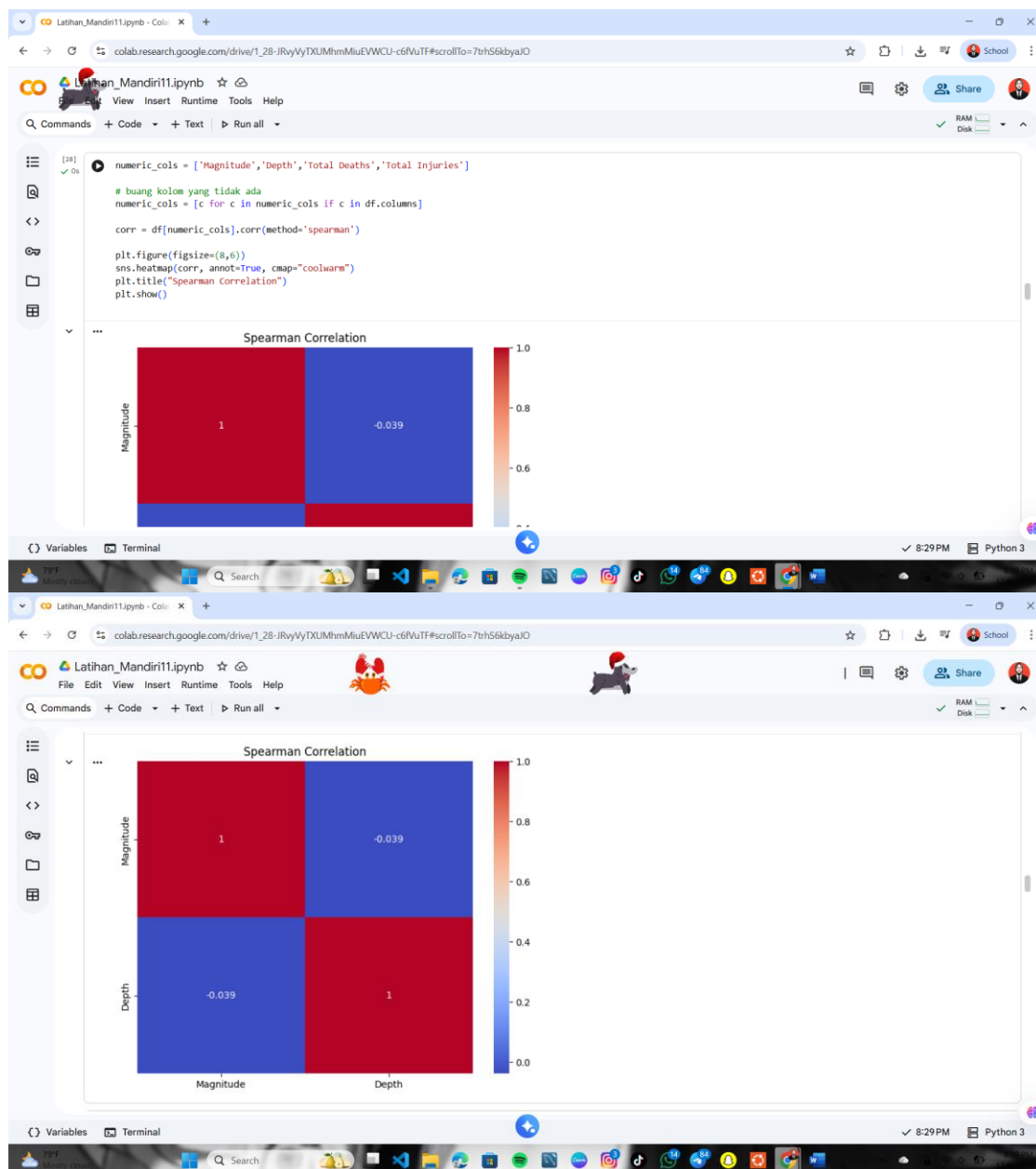
Output: Tidak ada baris yang mengandung nilai hilang (NaN) pada keempat kolom kunci (Latitude, Longitude, Depth, Magnitude). Tidak ada baris yang benar-benar duplikat dalam keseluruhan dataset.





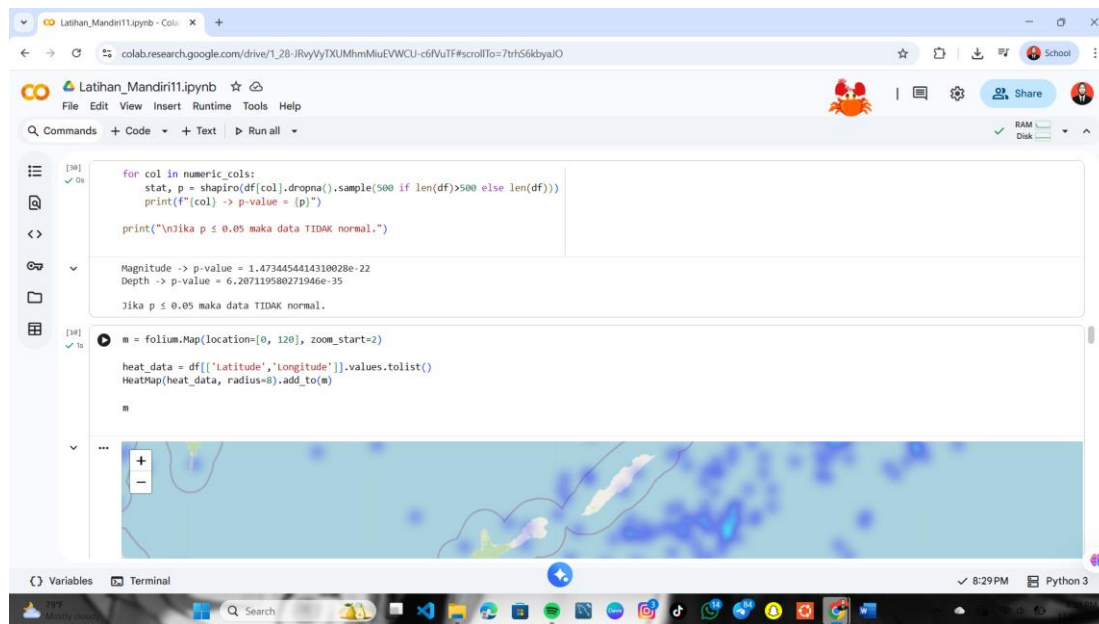
1. Menghasilkan dua plot visual:

- **Histogram Magnitude:** Menampilkan frekuensi nilai Magnitude gempa.
- **Histogram Kedalaman:** Menampilkan frekuensi nilai Kedalaman gempa.



1. **numeric_cols = ['Magnitude', 'Depth', 'Total Deaths', 'Total Injuries']**
Mendefinisikan list kolom yang ingin dianalisis korelasinya.
2. **numeric_cols = [c for c in numeric_cols if c in df.columns]**
Ini adalah langkah pembersihan list kolom
3. **corr = df[numeric_cols].corr(method='spearman')**
Menghitung matriks korelasi.

Output: memvisualisasikan korelasi antara Magnitude dan Depth.



1. **for col in numeric_cols**

Melakukan perulangan untuk setiap kolom dalam list numeric_cols (yang kemungkinan besar berisi Magnitude dan Depth).

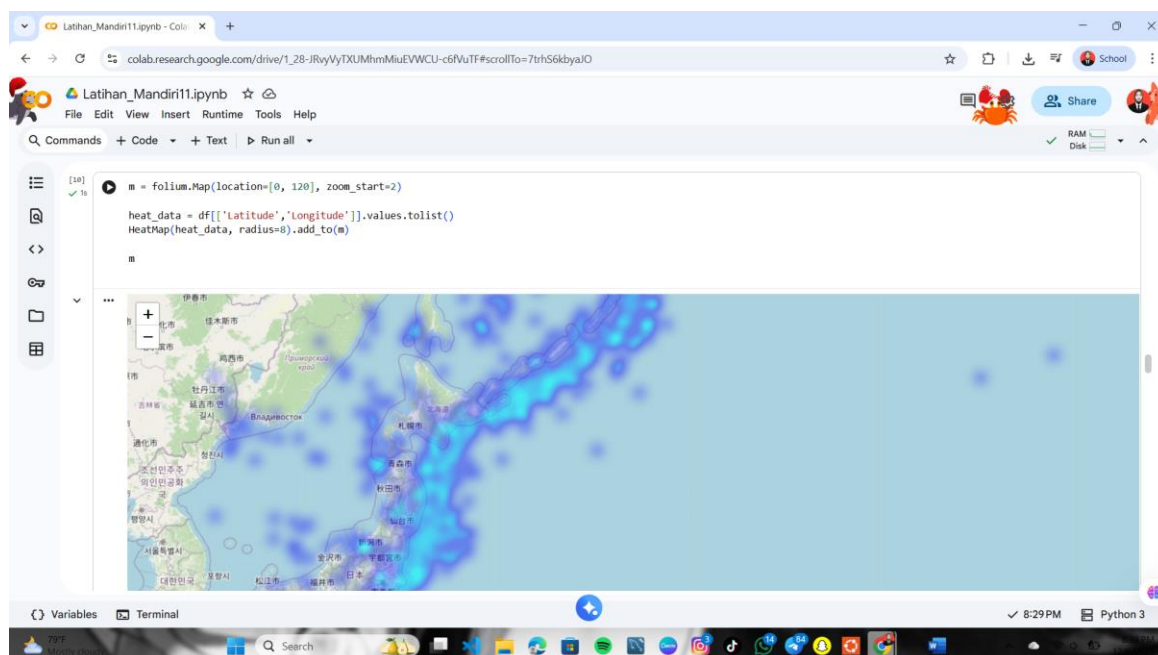
2. **stat, p = shapiro**

Melakukan Uji Shapiro-Wilk

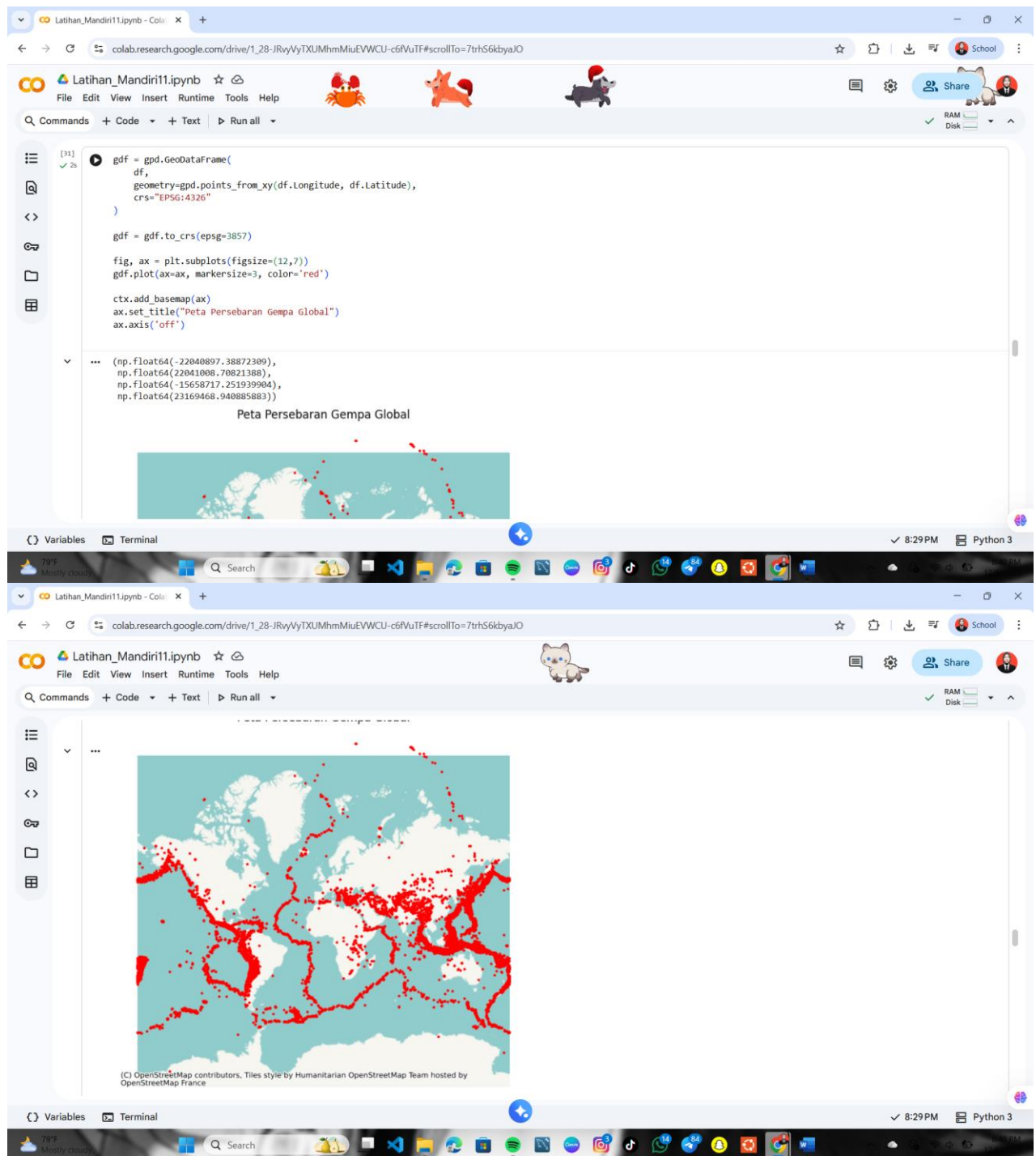
3. **print**

Mencetak nilai-p untuk setiap kolom.

Output: Karena kedua nilai-p jauh lebih kecil dari batas signifikansi umum (0.05), kita menolak hipotesis null (H0). Data Magnitude dan Depth TIDAK berdistribusi normal.



Membuat peta interaktif berpusat di dekat Indonesia/Samudra Pasifik, Mengambil koordinat Latitude dan Longitude yang sudah dibersihkan, Membuat lapisan Peta Panas (HeatMap) untuk memvisualisasikan konsentrasi lokasi gempa.



1. **gpd.GeoDataFrame()**
Membuat objek GeoDataFrame baru (gdf) dari DataFrame Pandas lama.
2. **geometry=gpd.points_from_xy**
Mengambil kolom Longitude dan Latitude untuk membuat objek geometris berupa titik (Point).
3. **crs="EPSG:4326"**
Menetapkan Sistem Referensi Koordinat (CRS) awal.
4. **gdf.to_crs(epsg=3857)**
Mengubah CRS dari WGS 84 (EPSG:4326) menjadi EPSG:3857 (Web Mercator Projection).
5. **fig, ax = plt.subplots**
Membuat objek gambar (fig) dan sumbu (ax) untuk plot menggunakan Matplotlib.

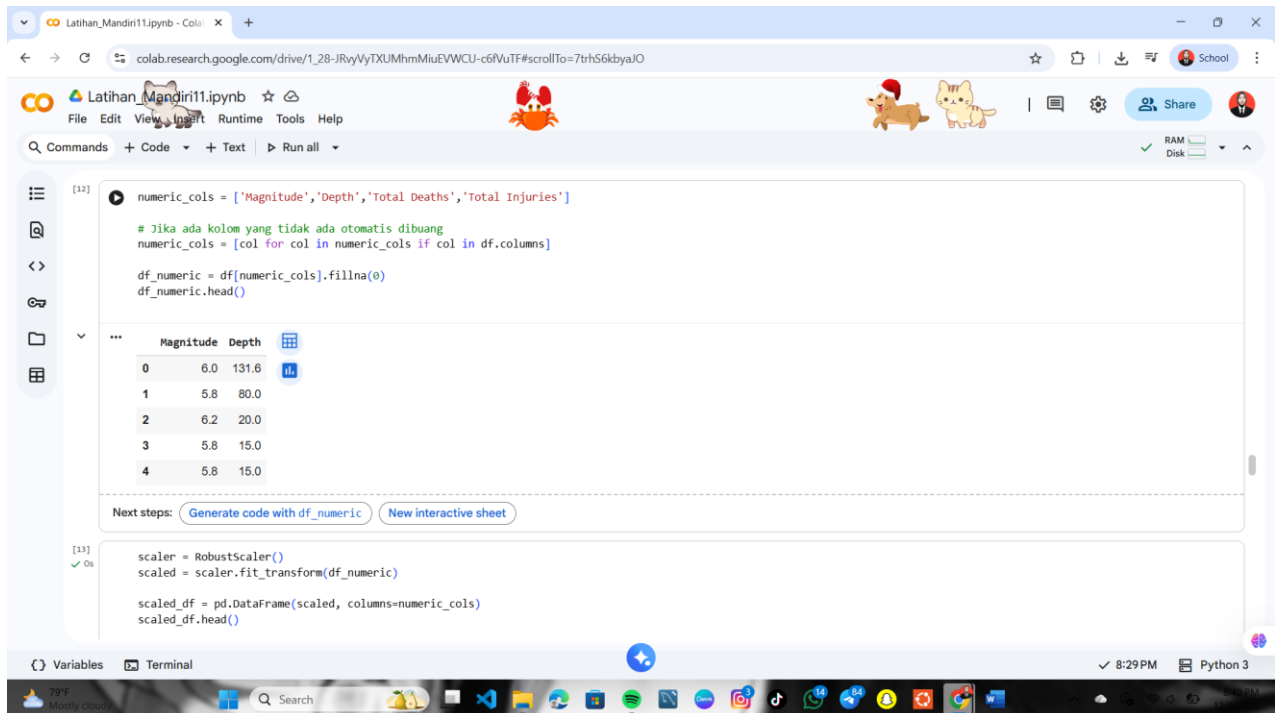
6. **gdf.plot**

Mem-plot GeoDataFrame.

7. **ctx.add_basemap(ax)**

Menggunakan pustaka contextily (ctx) untuk menambahkan peta dasar dari sumber online (misalnya, OpenStreetMap) ke plot.

Output: gambar peta statis



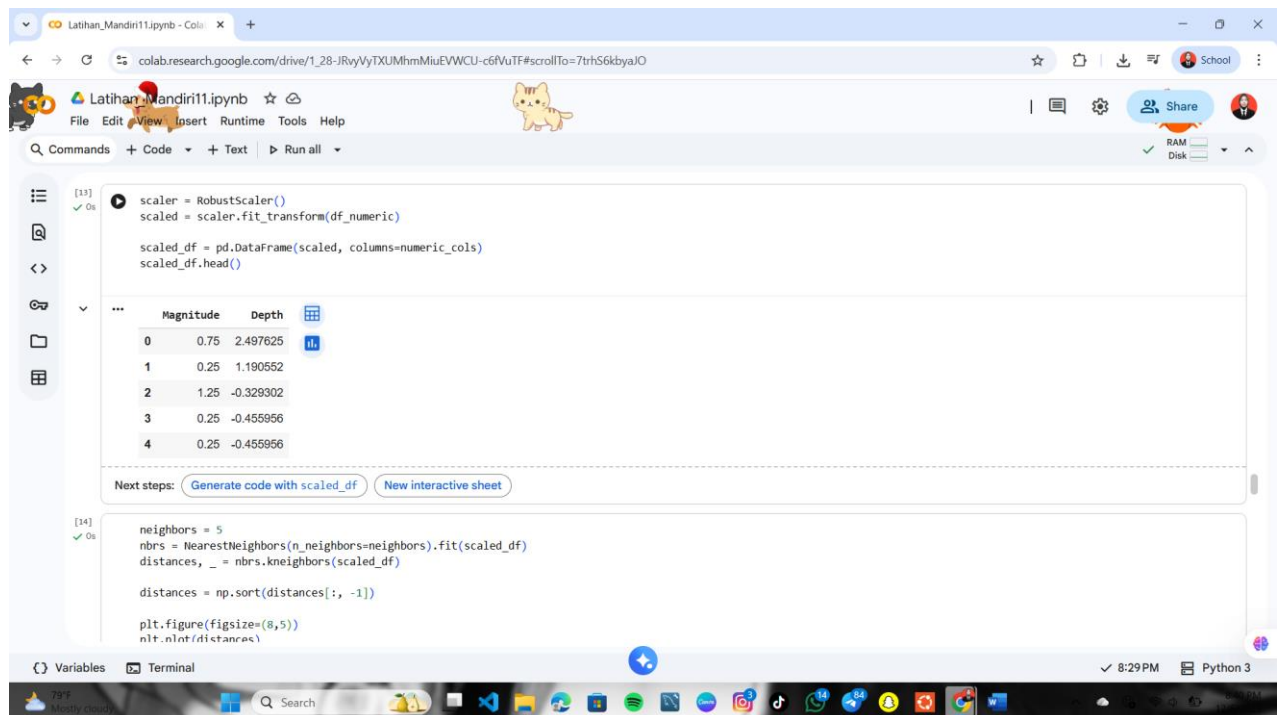
1. **numeric_cols = dan Pembersihan List**

Sama seperti langkah korelasi sebelumnya, list `numeric_cols` disaring untuk hanya menyisakan kolom yang benar-benar ada di `df`, yaitu `Magnitude` dan `Depth`.

2. **df_numeric.head()**

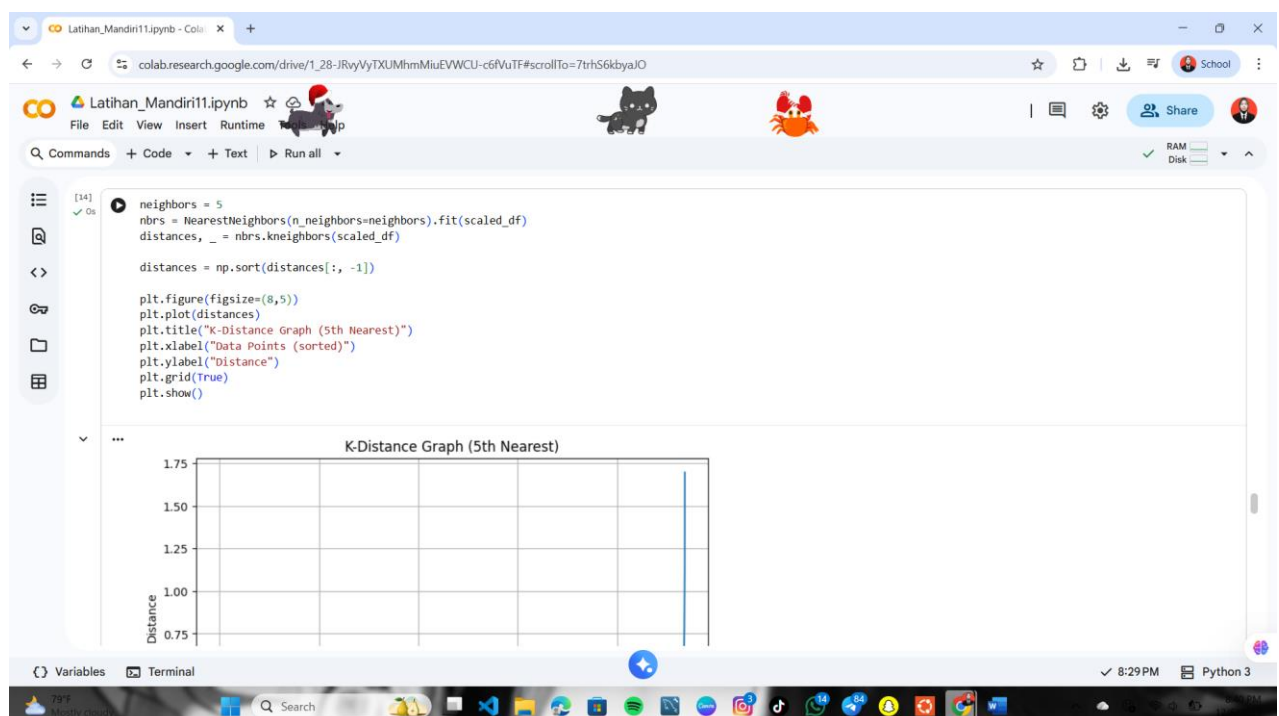
Menampilkan 5 baris pertama dari DataFrame numerik yang baru.

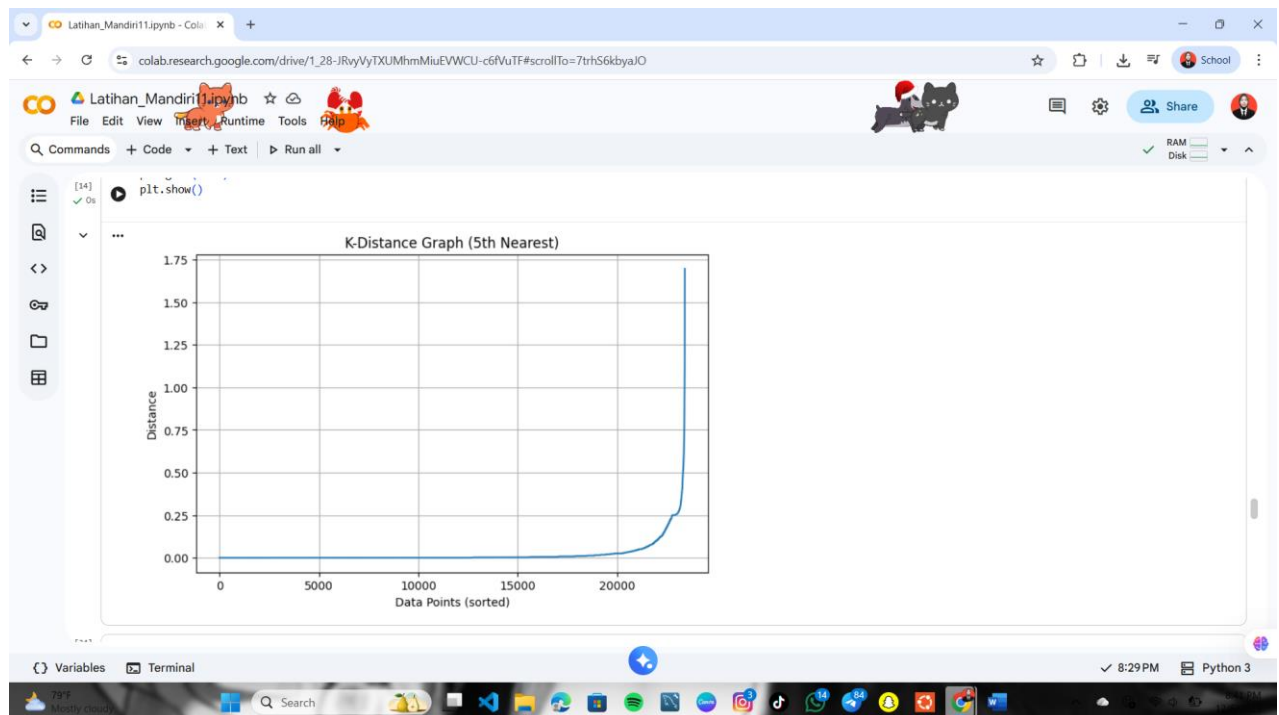
Output: tabel yang menampilkan DataFrame `df_numeric`.



Untuk menskalakan fitur numerik (Magnitude dan Depth) agar memiliki rentang yang seragam, yang berpusat di median nol. Penskalaan ini penting karena DBSCAN sensitif terhadap skala fitur.

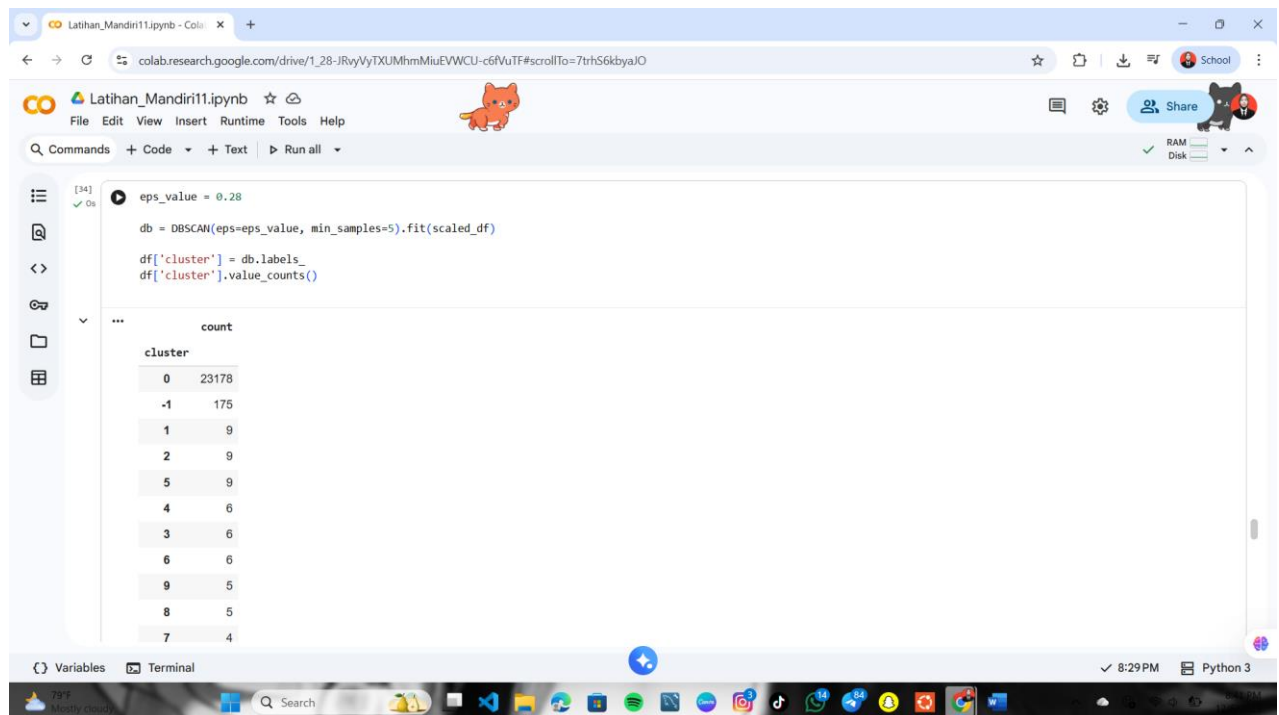
Output (Tabel scaled_df.head()): Menampilkan 5 baris pertama data yang sudah diubah oleh RobustScaler. Data sudah terpusat di sekitar nol (nilai median Magnitude, 6.0, kini menjadi 0.0 jika dicari di seluruh kolom), dan siap digunakan untuk perhitungan jarak.





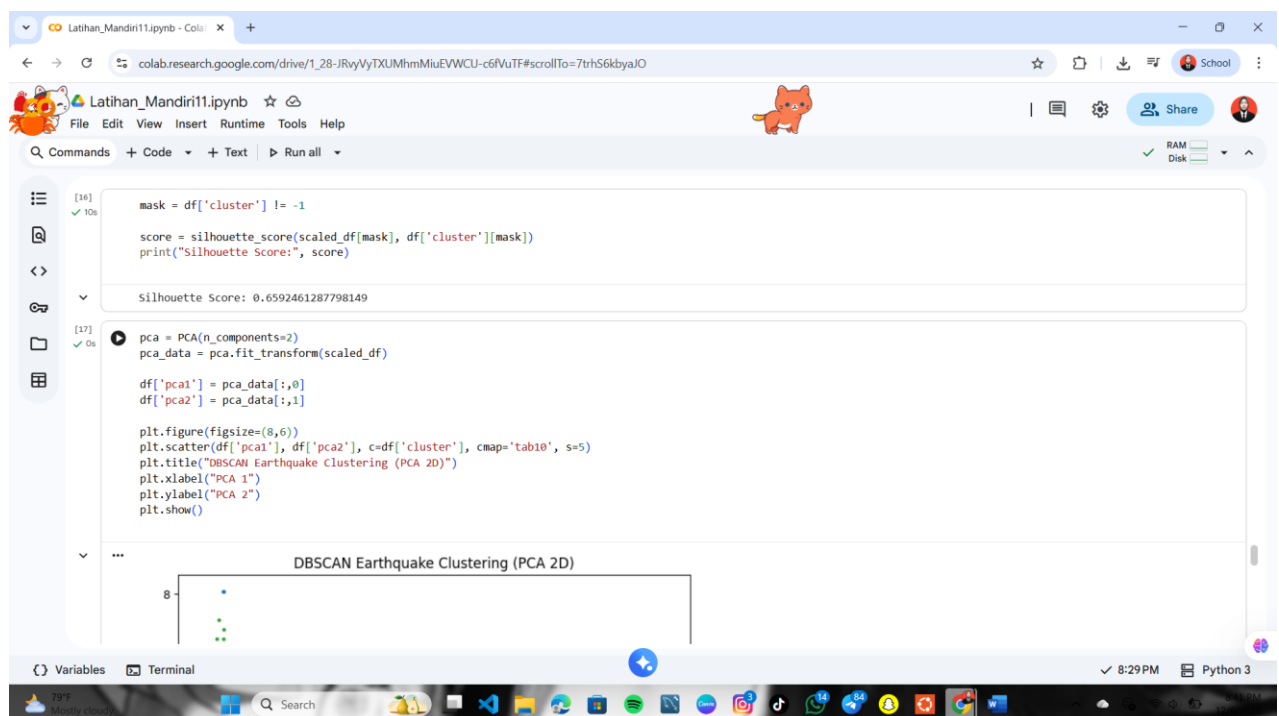
1. **neighbors = 5:**
Menetapkan k (jumlah tetangga terdekat) menjadi 5. Ini berarti kita mencari jarak ke tetangga ke-5.
2. **nbrs = NearestNeighbors(n_neighbors=neighbors).fit(scaled_df)**
Melatih model k-Nearest Neighbors pada data yang sudah diskalakan (scaled_df).
3. **distances, indices = nbrs.kneighbors(scaled_df)**
Menghitung jarak dari setiap titik data ke 5 tetangga terdekatnya.
4. **distances = np.sort(distances[:, -1], axis=0)**
Mengambil kolom jarak tetangga ke-5 dan mengurutkannya dari yang terkecil hingga terbesar.
5. **plt.plot(distances)**
Membuat plot garis dari jarak yang sudah diurutkan tersebut.

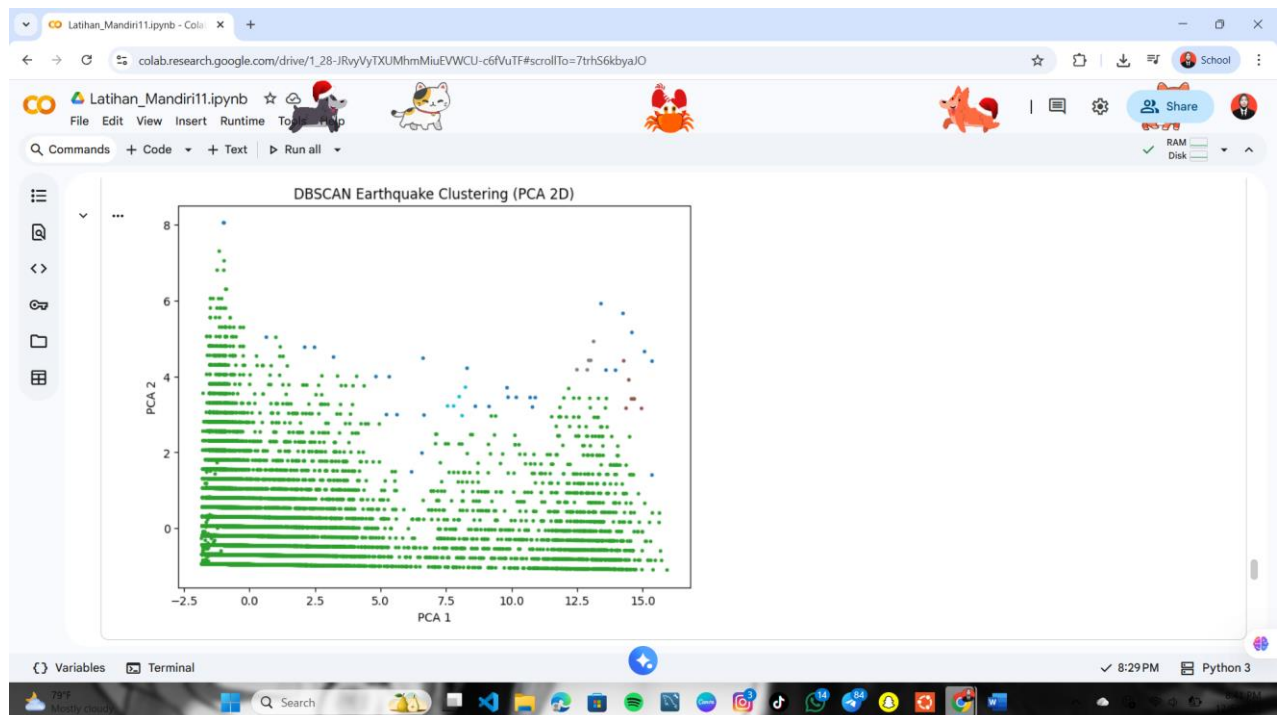
Output: Yang terlihat adalah grafik garis (plot k-distance) yang akan digunakan untuk mengidentifikasi titik tekukan (elbow) yang optimal.



1. **eps_value = 0.28**
Menetapkan parameter Epsilon atau eps menjadi 0.28.
2. **db = DBSCAN(eps=eps_value, min_samples=5).fit(scaled_df)**
Menginisialisasi model DBSCAN.
3. **df['cluster'] = db.labels_**
Hasil dari DBSCAN adalah label kluster untuk setiap titik data.
4. **df['cluster'].value_counts()**
Menghitung frekuensi kemunculan setiap label kluster yang ditemukan.

Output: tabel yang menampilkan hitungan (jumlah anggota) untuk setiap kluster yang ditemukan oleh DBSCAN.



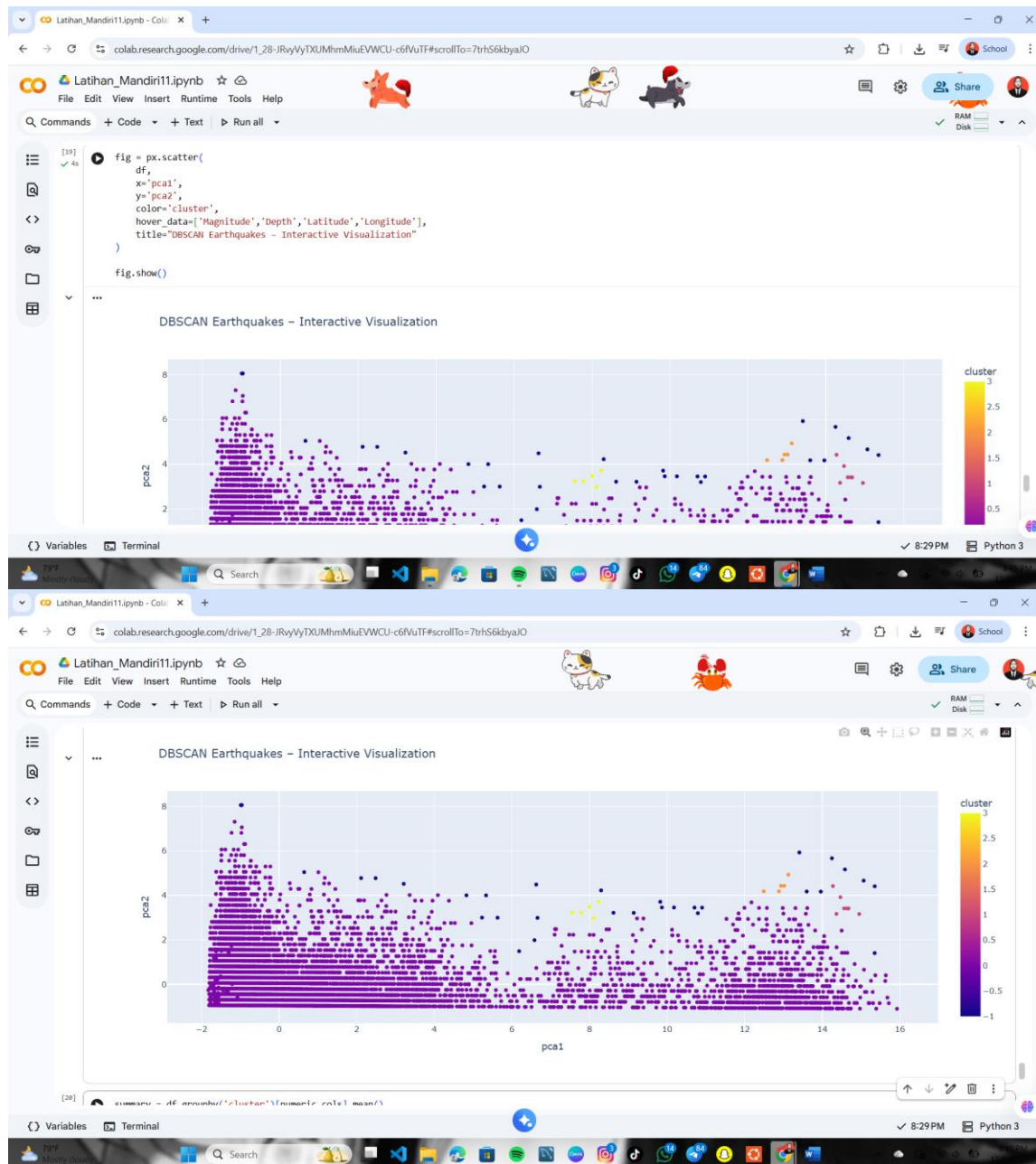


1. **mask = df['cluster'] != -1**
Membuat variabel boolean mask yang nilainya True hanya untuk titik data yang bukan noise (yaitu, titik yang memiliki label kluster > 0).
2. **score = silhouette_score(scaled_df[mask], df['cluster'][mask])**
Menghitung Silhouette Score menggunakan pustaka sklearn.metrics.

Output: Nilai 0.659. Nilai ini dianggap cukup baik dan menunjukkan bahwa kluster-kluster kecil yang ditemukan oleh DBSCAN (dengan label > 1), Meskipun sebagian besar data adalah noise (cluster = -1), kluster-kluster yang berhasil dibentuk memiliki kualitas internal yang solid.

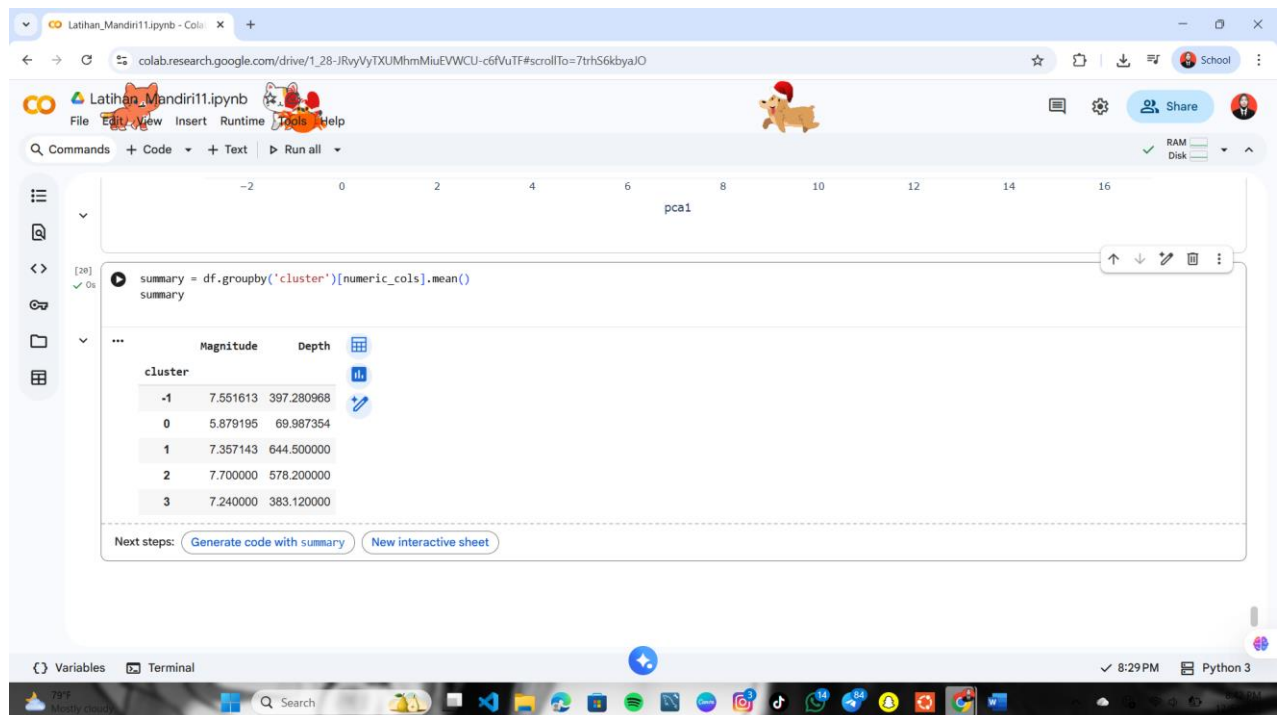
1. **pca = PCA(n_components=2):** Menginisialisasi model PCA untuk mereduksi dimensi menjadi 2 komponen utama.
2. **pca_data = pca.fit_transform(scaled_df):** Menerapkan PCA pada data yang sudah diskalakan (scaled_df).
3. **df['pca1'] = pca_data[:,0]** dan **df['pca2'] = pca_data[:,1]:** Menambahkan dua komponen PCA sebagai kolom baru ke DataFrame df.
4. **plt.scatter:** Membuat scatter plot (diagram sebar) untuk memvisualisasikan data

Output: scatter plot yang menunjukkan titik-titik data yang diwarnai berdasarkan kluster yang ditemukan.



1. **fig = px.scatter**
Membuat objek gambar interaktif (fig) dengan fungsi `px.scatter`.
2. **fig.show()**
Menampilkan plot interaktif Plotly.

Output: diagram sebar (scatter plot) yang divisualisasikan menggunakan Plotly.



1. **df.groupby('cluster')**
Mengelompokkan seluruh DataFrame df berdasarkan nilai unik yang ada di kolom cluster (label yang dihasilkan DBSCAN: -1, 0, 1, 2, 3, dst.).
2. **[numeric_cols]**
Memilih kolom-kolom yang ingin dihitung rata-ratanya, yaitu Magnitude dan Depth.
3. **.mean()**
Menghitung nilai rata-rata (average) untuk kolom Magnitude dan Depth di dalam setiap kluster.
4. **summary**
Menyimpan dan menampilkan hasilnya dalam DataFrame baru bernama summary.

Output: yang terlihat adalah tabel yang berisi rata-rata Magnitude dan Depth untuk setiap kluster yang teridentifikasi.

Interpretasi hasil clustering (jumlah cluster, noise, pola data)

Berdasarkan hasil penerapan algoritma DBSCAN pada dataset gempa bumi, diperoleh empat cluster utama dan satu kategori noise (label -1).

- **Cluster 0** memiliki rata-rata magnitudo sebesar **5.88** dan kedalaman rata-rata sekitar **69.99 km**. Cluster ini merepresentasikan gempa dangkal dengan kekuatan sedang yang merupakan kejadian gempa paling umum.
- **Cluster 1** memiliki rata-rata magnitudo **7.36** dan kedalaman rata-rata sangat dalam yaitu **644.50 km**. Cluster ini menggambarkan gempa dalam (deep-focus earthquakes) yang jarang terjadi namun memiliki ciri khas kedalaman ekstrem.
- **Cluster 2** memiliki rata-rata magnitudo **7.70** dan kedalaman rata-rata **578.20 km**, menunjukkan kelompok gempa kuat yang relatif dalam.
- **Cluster 3** memiliki rata-rata magnitudo **7.24** dan kedalaman rata-rata **383.12 km**, merepresentasikan kelompok gempa menengah–dalam dengan kekuatan besar.

- **Noise (-1)** memiliki magnitudo rata-rata **7.55** dan kedalaman rata-rata **397.28 km**, yang berarti data ini termasuk kejadian gempa ekstrem atau tidak mengikuti pola umum sehingga dikategorikan sebagai outlier oleh DBSCAN.

Pola yang terlihat menunjukkan bahwa gempa dangkal dengan magnitudo sedang cenderung membentuk cluster tersendiri, sedangkan gempa dengan kedalaman sangat dalam dan kekuatan besar membentuk cluster khusus. Titik-titik yang tidak memiliki cukup tetangga terdekat diklasifikasikan sebagai noise, yang umumnya merupakan kejadian gempa ekstrem atau langka.

Kesimpulan dari studi kasus clustering

Algoritma DBSCAN berhasil mengelompokkan data gempa bumi menjadi empat cluster utama berdasarkan kemiripan magnitudo dan kedalaman gempa. Cluster terbesar menunjukkan gempa dangkal dengan kekuatan sedang, sedangkan cluster lainnya menggambarkan gempa kuat dengan kedalaman sangat dalam. Data yang dikategorikan sebagai noise merepresentasikan kejadian gempa ekstrem atau tidak umum. Hal ini menunjukkan bahwa DBSCAN efektif dalam mengidentifikasi pola kepadatan data dan mendeteksi outlier pada data gempa bumi.

LINK GITHUB UPLOAD TUGAS : <https://github.com/Yurida26/Machine-Learning/tree/2a608d4212f9b401a8a909fa4b2c417758a0da15/Praktikum11>

